Guía: qué va en cada archivo (MVC en Arduino IDE)

Brazo robótico 3 GDL + pinza · ESP32 + PCA9685 · Laboratorio 1 de Fundamentos de Robótica

1. Una frase por capa

- **Modelo:** sabe qué es el brazo y dónde está (datos y matemática). No sabe que existe hardware.
- **Vista:** sabe cómo mostrar lo que dice el modelo (mensajes por Serial, JSON para la app). No decide ni mueve nada.
- **Controlador:** sabe qué hacer ante un comando o un evento (la rutina, la máquina de estados). Es el único que da órdenes.
- **Drivers:** saben cómo hablar con el hardware (PCA9685, sensor, WiFi). No saben de qué trata la tarea.
- **Config.h:** valores fijos que todos los demás leen.

Los drivers no pertenecen a ninguna de las letras de MVC: son la capa que toca el hardware y que usa el controlador.

2. Regla de dependencias (la más importante)

Las inclusiones van solo hacia abajo. Así puedes probar cada archivo sin los de arriba.

Archivo	Puede incluir	Nunca debe incluir
Config.h	nada	todo lo demás
Modelo_Cinematica	Config.h, math.h	Wire, Serial, drivers, vista, controlador
Modelo_Brazo	Config.h, Modelo_Cinematica	Serial, drivers, controlador
Vista_Telemetria	Modelo_Brazo, Serial	controlador, drivers
Driver_Servos, Driver_Sensor, Driver_WiFi	Config.h, librerías de hardware	modelo, vista, controlador
Controlador_Comandos	solo tipos de Arduino (String)	modelo, drivers

Archivo	Puede incluir	Nunca debe incluir
Controlador_Rutina	modelo, vista, drivers, Controlador_Comandos	nada de fórmulas D-H propias
PickAndPlace.ino	todos	lógica propia

Prueba rápida: si Modelo_Cinematica necesita un `#include <Wire.h>`, algo quedó en el lugar equivocado.

3. Ficha de cada archivo

Config.h

Responsabilidad: concentrar todo valor fijo o calibrable, para cambiarlo en un solo lugar.

Contiene:

- Pines y direcciones: SDA, SCL, dirección del PCA9685, pines del sensor.
- Geometría: eslabón 1 (94 mm), eslabón 2 (107 mm), altura del hombro sobre la mesa (pendiente de medir), tamaño del cubo (40 mm).
- Calibración: offset y signo de cada servo, límites mínimo y máximo, rango de pulsos PWM, ángulos de pinza abierta y cerrada.
- Puntos conocidos: recogida, depósito, home, poses de escaneo.
- Parámetros de movimiento: paso y pausa de la trayectoria, altura de aproximación.
- Umbral de detección del sensor.

No debe contener: funciones, variables que cambian en ejecución, ni contraseñas de WiFi (esas van en `secrets.h`, ignorado por Git).

Se prueba: compilando. No tiene lógica.

Driver_Servos (.h y .cpp)

Responsabilidad: traducir un ángulo en grados a una señal en un canal del PCA9685.

Contiene:

- Objeto del PCA9685, inicialización de I2C y de la frecuencia de 50 Hz.
- `iniciar()`, `escribirAngulo(canal, grados)` y `escribirTodos(grados[])`.
- La conversión de grados a valor PWM (la que ya tienes en `convertirAnguloAPwm`).

No debe contener: nada de cinemática ni de rutina. Conoce canales (0 a 3), no articulaciones: no sabe qué es un hombro ni un codo.

Se prueba: moviendo cada canal desde el Monitor Serie con un sketch temporal. Tu `Test_4servos_esp32.ino` ya cubre casi todo esto.

Driver_Sensor (.h y .cpp)

Responsabilidad: entregar una distancia confiable.

Contiene:

- `iniciar()`, `leerDistanciaCm()` y `leerDistanciaMediana(n)` (mediana de 5 lecturas para estabilidad).
- Manejo del timeout si no hay eco.

No debe contener: la decisión de si hay un bloque. Eso depende de la tarea y se hace en el controlador, comparando con el umbral de Config.h.

Se prueba: con el sketch de prueba del ultrasónico, midiendo distancias conocidas.

Modelo_Cinematica (.h y .cpp)

Responsabilidad: toda la matemática D-H. Son funciones puras: reciben números y devuelven números.

Contiene:

- Tipos `Punto` (x, y, z) y `Articulaciones` (q1, q2, q3).
- `cinematicaInversa(punto, q)`: devuelve falso si el punto está fuera de alcance.
- `cinematicaDirecta(q)` y, para el requisito de visualizar con D-H, `posicionesArticulaciones(q, salida[])`.
- Conversión articulación a servo y servo a articulación (offset y signo de Config.h).
- Comprobación de límites articulares.

No debe contener: `Serial`, `Wire`, `delay`, ni acceso a drivers.

Se prueba: sin el brazo. Compara sus resultados con `fk` e `ik_pos` de MATLAB para 3 o 4 puntos.

Modelo_Brazo (.h y .cpp)

Responsabilidad: ser la única fuente de verdad del estado del sistema.

Contiene:

- Ángulos actuales de los 4 servos y de las 3 articulaciones.
- Posición (x, y, z) de la pinza y de cada articulación (calculadas con la cinemática directa).
- Estado de la pinza (abierta o cerrada), nivel de pila y último valor del sensor.

- Fase actual de la rutina (REPOSO, ESCANEAR, RECOGER, ...) y bandera de error.
- Métodos como `actualizarDesdeServos(angulos)` y getters.

No debe contener: impresión, movimiento ni lógica de decisión. Solo guarda y calcula derivados.

Se prueba: asignándole ángulos a mano y revisando que la posición resultante coincida con MATLAB.

Vista_Telemetria (.h y .cpp)

Responsabilidad: presentar el modelo. Lee, nunca escribe en él.

Contiene:

- `iniciar()` (abre el Serial).
- `mostrarMenu()`, `mostrarEstado()` con texto legible.
- `generarJson()`: el estado en JSON para la app (ángulos, posición x, y, z, fase, sensor).
- `mensaje(tipo, texto)` para los avisos con formato [INFO], [AVISO] y [ERROR].

No debe contener: decisiones ni órdenes de movimiento. Si necesitas leer el Serial para recibir comandos, eso va en el controlador de comandos, no aquí.

Se prueba: pidiendo que muestre el estado con datos inventados.

Controlador_Comandos (.h y .cpp)

Responsabilidad: convertir texto recibido en una orden con significado.

Contiene:

- Un `enum class Comando { NINGUNO, INICIAR_RUTINA, IR_HOME, DETENER, ESTADO, ... }`.
- `leerSerie()` y, más adelante, una función que recibe el texto que llega por WiFi.
- La validación (ignorar líneas vacías o inválidas, como ya haces en `procesarPuertoSerie`).

No debe contener: la ejecución de las órdenes. Solo las reconoce y las entrega.

Se prueba: escribiendo comandos válidos e inválidos y mirando qué Comando devuelve.

Controlador_Rutina (.h y .cpp)

Responsabilidad: ejecutar la tarea: la máquina de estados de recoger y apilar.

Contiene:

- `ordenar(comando)`: recibe una orden y cambia de fase.

- `actualizar()`: se llama en cada `loop()` y avanza la rutina.
- La secuencia: ESCANEAR, APROXIMAR, DESCENDER, AGARRAR, ELEVAR, TRANSPORTAR, DEPOSITAR, LIBERAR, RETIRAR, volver a home.
- El movimiento en línea recta: pide a Modelo_Cinematica los ángulos de cada punto intermedio y los envía a Driver_Servos.
- La decisión de detección: compara la distancia del sensor con el umbral.
- Las protecciones: puntos fuera de alcance, límites, errores.
- La actualización del modelo y los avisos a la vista tras cada paso.

No debe contener: fórmulas de cinemática (las llama), ni `pca9685.setPWM` (lo hace el driver), ni `Serial.println` (lo hace la vista).

Se prueba: primero sin cubo, después con cubo, y repitiendo la rutina unas 10 veces para medir repetibilidad.

Driver_WiFi (.h y .cpp), se agrega en el paso de WiFi

Responsabilidad: levantar el punto de acceso y el servidor HTTP del ESP32.

Contiene: las rutas (por ejemplo, una para enviar comandos y otra para pedir el estado). Pasa el texto del comando al controlador de comandos y devuelve el JSON que prepara la vista.

No debe contener: lógica de la rutina. Las credenciales van en `secrets.h`.

Este archivo no estaba en la lista original de 9; se suma cuando llegues a la comunicación.

PickAndPlace.ino

Responsabilidad: crear los objetos y conectarlos.

Contiene: las instancias globales, `setup()` (llama a `iniciar()` de cada módulo) y `loop()` (lee comandos, los entrega y llama a `actualizar()`).

No debe contener: ninguna lógica propia. Si crece, algo debe moverse a su archivo.

4. Interfaces de ejemplo (solo declaraciones)

Config.h:

```
#pragma once
constexpr float LONGITUD_ESLABON_1_MM = 94.0f;
constexpr float LONGITUD_ESLABON_2_MM = 107.0f;
constexpr float ALTURA_HOMBRO_MM      = 0.0f;   // PENDIENTE: medir
constexpr float ALTURA_CUBO_MM        = 40.0f;
constexpr int   CANTIDAD_SERVOS       = 4;
```

Modelo_Cinematica.h:

```
#pragma once
struct Punto { float x, y, z; };
struct Articulaciones { float q1, q2, q3; };   // grados

bool cinematicaInversa(const Punto& destino, Articulaciones& q);
Punto cinematicaDirecta(const Articulaciones& q);
```

Driver_Servos.h:

```
#pragma once
#include "Config.h"

class DriverServos {
public:
    void iniciar();
    void escribirAngulo(int canal, float grados);
    void escribirTodos(const float grados[CANTIDAD_SERVOS]);
private:
    int gradosAPwm(float grados) const;
};
```

Controlador_Rutina.h:

```
#pragma once
#include "Modelo_Brazo.h"
#include "Vista_Telemetria.h"
#include "Driver_Servos.h"
#include "Driver_Sensor.h"
#include "Controlador_Comandos.h"

class ControladorRutina {
public:
    ControladorRutina(ModeloBrazo& modelo, VistaTelemetria& vista,
                      DriverServos& servos, DriverSensor& sensor);
    void iniciar();
    void ordenar(Comando comando);
    void actualizar();
```

```
private:
    ModeloBrazo& modelo_;
    VistaTelemetria& vista_;
    DriverServos& servos_;
    DriverSensor& sensor_;
};
```

PickAndPlace.ino:

```
#include "Modelo_Brazo.h"
#include "Vista_Telemetria.h"
#include "Driver_Servos.h"
#include "Driver_Sensor.h"
#include "Controlador_Comandos.h"
#include "Controlador_Rutina.h"

ModeloBrazo modelo;
VistaTelemetria vista(modelo);
DriverServos servos;
DriverSensor sensor;
ControladorComandos comandos;
ControladorRutina rutina(modelo, vista, servos, sensor);

void setup() {
    servos.iniciar();
    sensor.iniciar();
    vista.iniciar();
    rutina.iniciar();
}

void loop() {
    Comando c = comandos.leerSerie();
    if (c != Comando::NINGUNO) rutina.ordenar(c);
    rutina.actualizar();
}
```

Los módulos reciben sus dependencias en el constructor, como referencias. Así evitas variables globales que se usan desde varios archivos.

5. Recorrido de un caso: el usuario presiona AUTO

1. La app envía una petición al ESP32 y Driver_WiFi recibe el texto del comando.
2. Controlador_Comandos lo convierte en `Comando::INICIAR_RUTINA`.

3. El `.ino` se lo entrega con `rutina.ordenar(c)` y la rutina pasa de REPOSO a IR_HOME.
4. En cada `loop()`, `rutina.actualizar()` decide el siguiente paso, pide a Modelo_Cinematica los ángulos de un punto (x, y, z), los envía a Driver_Servos y actualiza Modelo_Brazo.
5. Vista_Telemetria lee Modelo_Brazo y prepara el texto o el JSON. La app lo consulta con una petición de estado.
6. En la fase ESCANEAR, Driver_Sensor entrega la distancia, y el controlador la compara con el umbral de Config.h para decidir si recoge o si omite el punto.

6. Si dudas dónde va algo

Código	Archivo
Longitud de un eslabón, un pin, un umbral	Config.h
Ley de cosenos, atan2, matrices D-H	Modelo_Cinematica
Ángulo actual del codo, nivel de pila	Modelo_Brazo
`pca9685.setPWM(...)`	Driver_Servos
`pulseIn(...)` del ultrasónico	Driver_Sensor
`Serial.println(...)`	Vista_Telemetria
Interpretar el texto `r` o `h`	Controlador_Comandos
Si el sensor ve algo, bajar a recoger	Controlador_Rutina
`delay(...)`	solo en Controlador_Rutina, y mejor evitarlo (ver abajo)
Contraseña de WiFi	secrets.h

7. Detalles del Arduino IDE

- Pon `#pragma once` en la primera línea de cada `.h`.
- Incluye tus archivos con comillas (`#include "Config.h"`) y las librerías con `<...>`.
- Mantén la carpeta plana: todos los archivos en la raíz del sketch.
- Separa los módulos en `.h` y `.cpp`, no en varias pestañas `.ino`.

- Sobre `delay()`: empieza con movimientos bloqueantes dentro de Controlador_Rutina, como tu sketch original. Hazlos no bloqueantes con `millis()` solo si te sobra tiempo. Con esta estructura, ese cambio no afecta a los demás archivos.

8. Orden de creación sugerido

1. Config.h y Driver_Servos.
2. Calibración y Modelo_Cinematica (se prueba contra MATLAB, sin el brazo).
3. Modelo_Brazo.
4. Movimiento a un punto en línea recta (dentro de Controlador_Rutina) y medición del error.
5. La rutina completa, sin cubo y luego con cubo.
6. Driver_Sensor y la fase ESCANEAR.
7. Vista y Controlador_Comandos por Serial, y después Driver_WiFi.
8. La app Android.